ALMA: Meta-learning Agentic Memory Designs

Yiming Xiong, Shengran Hu, Jeff Clune · University of British Columbia · Vector Institute · arXiv 2602.07755v1

HarnessRSI 75 ★★★

evidence: improver

Paper

Code

1 What changes, what is kept

One memory design = one Python module (code is the search space)

Layers	sub-modules with own update() / retrieve()	**Database**	optional per layer: vector, graph, dict
Interface	general_update() general_retrieve()	**Fixed agent**	GPT-5-nano, never edited by the search

1 Archive of designs
every evaluated design stays; parents drawn by success, fewer visits favoured; up to 5 per step

2 Meta Agent (GPT-5) plans
reads parent code, stratified logs and success rate, then writes a modification plan

3 Implement + trial run
new design written in code; 5-task trial; debug up to 3 times

4 Score by deployment
collect memory without retrieval, then deploy: success rate, mean of 3 runs

5 Add to archive
design + logs stored; no reject step

11×

after 11 steps (43 designs): best on learning set → test split
weights never change; the kept artifact is code

2 Headline: clear only on ALFWorld

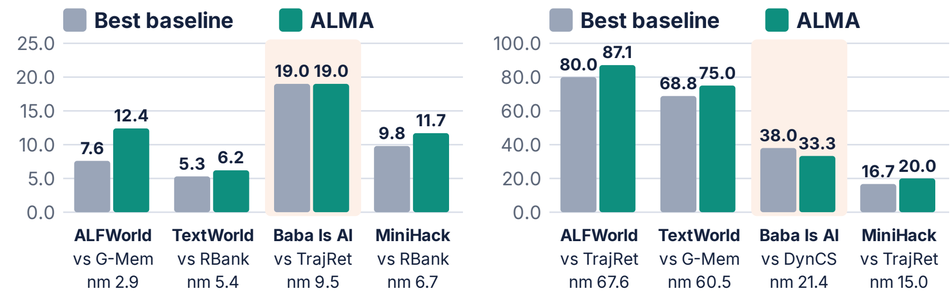


Table 1, static mode, mean of 3 runs; best or tied in 7 of 8 columns. Margin ≈ 6.8‡ / 3.6‡ combined SE on ALFWorld (nano / mini). Baseline = best manual design there; nm = no memory. Shaded: Baba Is AI (tie / loss).

4 Where the gain is absent or thin

- **Baba Is AI, mini:** ALMA 33.3 ± 2.4 vs Dynamic Cheatsheet 38.0 ± 6.3 (4.7‡ behind); nano tie at 19.0. Abstract: “on all benchmarks”
- Within noise: TextWorld nano 6.2 ± 1.7 vs 5.4 ± 1.0 (no memory); MiniHack nano 11.7 ± 2.2 vs 9.8 ± 1.7; TextWorld mini 75.0 ± 2.3 vs 68.8 ± 4.7
- Baba values sit on a 1/42 grid (14 tasks) but 52 tasks give 13 (paper-internal mismatch)
- Best of 43 designs picked on a small learning metric (ALFWorld: 15 tasks × 3 runs)

3 Controls that decide if the gain is real

Open-ended vs greedy (Table 2)

ALFWorld · 43 designs	greedy	ALMA	gap
GPT-5-nano	11.9 ± 0.5	12.4 ± 0.5	0.5‡
GPT-5-mini	77.1 ± 0.8	87.1 ± 1.4	10.0‡

Varies the parent-sampling rule. Greedy mini 77.1 is 2.9‡ below Trajectory Retrieval 80.0 (≈ 1.8‡ SE). Not a same-start control: own initial design, 43 steps vs 11.

What the paper does and does not control

- **yes** held-out test split; SE over 3 deployment runs
- **yes** greedy ablation, same 43 designs (ALFWorld)
- **yes** nano → mini (effort also changes); shift 84.1%
- **no** repeated searches (one reported per benchmark)
- **no** baselines searched or tuned; their budget
- **no** search cost ($0.09 = memory cost only)
- **no** ablations: sampling parts, Meta Agent, abstraction
- **no** cross-benchmark transfer; leakage audit

Baselines are adapted to the static protocol (top-1 retrieval, frozen cheatsheet, ground-truth feedback); ALMA is searched under it. Split drawing (random or ordered) is not stated; TextWorld mixes two game types.

Takeaway

A Meta Agent writing memory code clearly beats four hand-built memories on ALFWorld; elsewhere it is ahead by less than about two SE or behind (Baba Is AI, mini). The paper reports one search per benchmark, baselines with no search of their own, and a greedy-search control only on ALFWorld.

Our read · HarnessRSI

Score 75 · ★★★ · evidence level: improver

Reports: held-out test, 3-run SE, model transfer, distribution shift, greedy ablation, memory cost.
Does not report: repeated searches, search cost, searched baselines, leakage check.